

GNN-Guided Retrieval for Knowledge Graph Question Answering: Architectures, Evidence Subgraphs, and End-to-End Analysis

Anonymous authors

Paper under double-blind review

Abstract

Graph neural retrieval offers a way to connect language-model question answering with explicit relational evidence, but its effectiveness depends on both the retriever and the evidence supplied to the generator. We present an independent implementation and empirical reassessment of this workflow, inspired by GNN-RAG. On a shared, filtered WebQSP evaluation set, we compare six graph neural retrieval architectures, investigate shortest-path and rooted prize-collecting Steiner tree (PCST) evidence construction, and evaluate two language models. NBFNet achieves the strongest mean retrieval results, including Hits@10 of 0.921 and gold-answer coverage of 0.877. PCST reduces the evidence from approximately 94 triples to 13–15 triples per question and reduces total language-model token use by approximately 78–80%. These reductions largely preserve answer-entity coverage but lower final answer quality. Semantic edge costs improve the compactness–quality trade-off over constant costs, while the union of shortest paths produces the best final answers. Conditional analysis shows that the language models omit at least one correct answer in approximately 21–24% of fully covered shortest-path contexts, rising to approximately 47–48% with semantic PCST. The results, therefore, identify the evidence-to-answer transition as a substantial bottleneck and show why entity coverage alone is insufficient for evaluating the usefulness of graph evidence.

1 Introduction

Knowledge graphs organize facts as entities and typed relations, providing an explicit structure for representing and accessing knowledge (Hogan et al., 2021). Answering a natural-language question over such a graph requires more than identifying entities with similar names. The system must connect the entities mentioned in the question to possible answers through relations that express the requested meaning. A correct answer may therefore depend on several intermediate facts, while nearby entities connected through other relations may be irrelevant. This makes the selection of useful evidence a central part of knowledge graph question answering (KGQA).

Retrieval-augmented generation extends a language model with external information that can support the interpretation and answering of a question (Lewis et al., 2020). When the external source is a knowledge graph, retrieval operates over entities and relational facts rather than passages of prose. GNNs can propagate information through this structure and rank entities according to their relevance as candidate answers. Candidate rankings alone, however, do not provide an explicit explanation of how those entities relate to the question. An additional evidence-construction stage can therefore extract the connecting facts and present the language model with a smaller, structured context from which to generate the final answer.

GNN-RAG provides the main motivation for the workflow examined in this study. Its pipeline uses a graph neural retriever to identify candidate answers, shortest paths to recover the relational evidence connecting them to the question entities, and a language model to generate the final answer from that evidence (Mavromatis & Karypis, 2025). This division of responsibilities is attractive because retrieval and generation address distinct

parts of the task. At the same time, the modular design makes the final outcome dependent on choices within each stage.

We examine these dependencies through an independent implementation and systematic empirical reassessment of GNN-guided KGQA. The study compares six retrieval architectures, then carries the strongest retriever into an investigation of evidence construction and final generation. In addition to a union of shortest paths, we consider rooted prize-collecting Steiner trees (PCST), motivated by their use for compact graph retrieval in G-Retriever (He et al., 2024). Candidate ranks determine node prizes, while constant or question-dependent semantic costs determine the price of connecting them.

The investigation addresses three research questions:

- **RQ1:** How does the retrieval architecture affect the ranking and coverage of correct answers?
- **RQ2:** How does evidence construction affect context compactness and information preservation?
- **RQ3:** How does the language model affect final-answer quality and the use of available evidence?

All three questions are evaluated within a shared pipeline, allowing downstream comparisons to reuse the same upstream predictions.

Our contributions are a modular implementation with separately measurable retrieval, evidence, and generation stages; a comparison of six adapted retrievers under shared data and training controls; an assessment of shortest-path evidence against two rooted PCST cost strategies; and an analysis connecting final answer errors to the availability of correct entities in the supplied context. We reassess the underlying methodology and its extensions rather than claim numerical replication of the complete GNN-RAG experimental setting.

The central finding is that strong entity retrieval and high context coverage do not guarantee complete answers. PCST achieves substantial compression with small changes in gold-entity coverage, yet final recall and F1 fall. This motivates evaluating graph evidence through both its retained entities and its usefulness to the downstream generator.

2 Background and Related Work

Graph neural reasoning. Graph neural architectures differ in how they represent local graph structure. GraphSAGE learns node representations by aggregating neighboring states (Hamilton et al., 2017), while R-GCN incorporates relation types through relation-dependent transformations and basis decomposition (Schlichtkrull et al., 2018). HGT further uses type-dependent projections and attention to model heterogeneous interactions (Hu et al., 2020). Although these mechanisms offer increasingly relation-aware aggregation, their original formulations are not interchangeable KGQA systems. Their input representations, conditioning mechanisms, and answer-scoring heads must therefore be specified when adapting them for retrieval.

Other architectures more directly model a reasoning process. ReaRev derives instructions from the question and revises them using information collected from the graph (Mavromatis & Karypis, 2022). NBFNet formulates link prediction through learned path composition and aggregation, inspired by the Bellman–Ford algorithm (Zhu et al., 2021). Our NBFNet adaptation replaces the query-relation input with a natural-language question representation and scores possible answers relative to the supplied seed entities. The comparison therefore covers shared aggregation, relation-specific computation, adaptive instructions, and path propagation.

Graph retrieval for generation. GNN-RAG connects learned candidate retrieval with shortest-path evidence for language-model reasoning (Mavromatis & Karypis, 2025). G-Retriever studies retrieval-augmented generation over textual graphs and uses a PCST-based formulation to obtain a compact relevant subgraph (He et al., 2024). These approaches motivate the two evidence families considered here. The classical prize-collecting formulation balances the value of selected nodes against their connection costs, allowing a candidate to be omitted when its prize does not justify the cost of including it (Goemans & Williamson, 1995).

Our evidence construction differs from a direct reproduction of G-Retriever: prizes come from GNN candidate ranks, the question entities anchor the solution, and the output is a set of original knowledge-graph triples. We compare this construction with shortest-path union while holding the retriever fixed. The contribution is thus an empirical examination of methodological choices within a known retrieval-to-generation pattern, with measurements at intermediate stages as well as at the final answer.

3 Methodology

3.1 Problem formulation and common workflow

For each question q , let $\mathcal{G}_q = (V_q, E_q)$ denote its supplied local graph, where every edge is represented as a directed triple (u, r, v) . Let $S_q \subseteq V_q$ be the set of valid seed entities supplied with the question, and let A_q denote its labeled answer set. A retriever R_θ assigns a score to each node in V_q , from which a ranked candidate list C_q is constructed. An evidence builder B then selects a set of triples $H_q \subseteq E_q$, and a language model L produces the predicted answer set P_q :

$$C_q = \text{Select}(R_\theta(q, \mathcal{G}_q, S_q)), \quad H_q = B(\mathcal{G}_q, S_q, C_q), \quad P_q = L(\text{textualization}(q, H_q)). \quad (1)$$

The arguments of R_θ define the common retriever interface and individual architectures may use different subsets of the available question and seed information.

Figure 1 shows the complete workflow. Each stage persists its output, so evidence strategies can reuse a trained retriever and language models can consume the same selected facts. The graph defines the answer-search space: we do not retrieve directly from the complete database or perform entity linking in this study.

3.2 Retrieval architectures

Shared aggregation and learned question conditioning. The GraphSAGE baseline is a weighted adaptation of neighborhood aggregation (Hamilton et al., 2017). Entity embeddings are projected into a hidden space and messages use shared transformations weighted by the cosine similarity between question and relation embeddings. At the end, a node classifier produces answer logits. Our extension, Advance GraphSAGE, on the other hand, replaces the fixed similarity weight with a learned gate:

$$\alpha(q, r) = \text{sigmoid}(\text{MLP}[\tilde{\mathbf{q}} \parallel \tilde{\mathbf{r}} \parallel (\tilde{\mathbf{q}} \odot \tilde{\mathbf{r}})]). \quad (2)$$

The extension additionally adds reverse edges, residual updates, layer normalization, and a classifier conditioned on $[\mathbf{h}_v \parallel \tilde{\mathbf{q}} \parallel (\mathbf{h}_v \odot \tilde{\mathbf{q}})]$. These changes are evaluated together as one configuration, not as individual ablations.

Relation-aware aggregation. R-GCN uses relation-specific transformations with basis decomposition (Schlichtkrull et al., 2018), while HGT uses relation-dependent multi-head attention (Hu et al., 2020). Because the prepared graphs have one node type, HGT’s node-type transformations become shared projections while its relation-dependent operations are retained. In the evaluated implementations, these two models use categorical relation identifiers and node features without direct question conditioning in message passing or their classifier. Consequently, the experiment compares complete retrieval adaptations rather than isolating relation-specific computation alone.

Adaptive instructions and path propagation. ReaRev uses a frozen MiniLM encoder for question tokens and relation text, generates multiple instructions, and alternates graph reasoning with instruction revision (Mavromatis & Karypis, 2022; Wang et al., 2020). Its final output is a probability distribution over graph nodes. NBFNet instead initializes seed nodes with a projected question vector and other nodes with zero, retaining this boundary signal during propagation. In layer ℓ , a relation transformation depends on the question and modulates the source state:

$$\mathbf{w}_r^{(\ell)}(\mathbf{q}) = W_r^{(\ell)}\mathbf{q} + \mathbf{b}_r^{(\ell)}, \quad \mathbf{m}_{u,r,v}^{(\ell)} = \mathbf{h}_u^{(\ell)} \odot \mathbf{w}_r^{(\ell)}(\mathbf{q}). \quad (3)$$

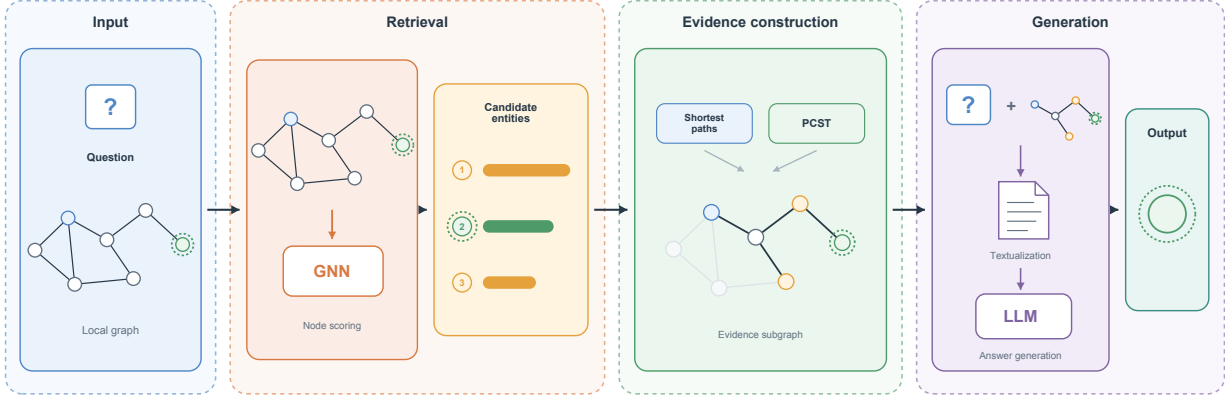


Figure 1: System overview, from the question and local graph through GNN candidate retrieval, evidence construction, triple textualization, and answer generation. Blue marks a seed, orange marks candidate answers, and the green dotted ring identifies an illustrative gold answer. The gold annotation is explanatory and is not supplied to the system.

This adaptation uses a DistMult-style message operator and principal neighbourhood aggregation (PNA), followed by residual updates and question-conditioned node scoring (Yang et al., 2015; Corso et al., 2020). Forward and reverse relations have separate parameters. Architectural details and training configurations are detailed in Appendix A.

3.3 Candidate selection

Nodes are ranked by their output probabilities. We retain nodes with probability at least $\tau = 0.7$, complete the list with the highest-ranked remaining nodes if fewer than $k = 10$ were retained, and cap the result at $K = 15$. If the graph has fewer nodes, all available nodes can be retained. This rule applies to every retriever.

3.4 Evidence-subgraph construction

Union of shortest paths. For each reachable candidate, we collect all paths of minimum distance from the seed set and take the union of their triples. Structural traversal treats adjacency as undirected, while emitted facts retain their original relation and direction. Multiple equally short paths can therefore contribute additional evidence, and repeated triples are removed. This construction follows the use of shortest-path evidence motivating GNN-RAG (Mavromatis & Karypis, 2025).

Rooted PCST. Let $c_1, \dots, c_m$ be the valid, reachable candidates in ranked order. Candidate c_i receives prize $\pi(c_i) = m - i + 1$, while other nodes have zero candidate prize. We seek a seed-rooted structure with objective

$$\max_T \left[\sum_{v \in V_T} \pi(v) - \sum_{e \in E_T} c(e) \right], \quad c_{\text{const}}(e) = \lambda, \quad c_{\text{sem}}(e) = \max\{\epsilon, \lambda[1 - \cos(\mathbf{q}, \mathbf{r}_e)]\}. \quad (4)$$

The rooted PCST solver is used as an approximate optimizer, so its output is not assumed to be an exact maximizer of the objective. In the semantic-cost variant, edge costs are derived from question and relation embeddings, with $\epsilon = 10^{-6}$ ensuring that they remain positive. Root construction depends on the number of valid seeds: a single seed serves directly as the root, whereas multiple seeds are connected to a temporary root through zero-cost edges and assigned sufficiently large additional prizes to enforce their inclusion. This auxiliary structure is removed before textualization, allowing the final evidence to consist of several components anchored at different seeds.

Both PCST variants operate on original facts represented as undirected structural edges. Parallel relations remain separate possibilities, and artificial reverse edges used by a retriever are not emitted as new facts. Selected edges map back to their original triples. Unlike shortest-path union, PCST can omit a reachable candidate when retaining it is not worthwhile under the prize-cost objective.

3.5 Answer generation

For final-answer generation, the selected evidence is serialized in the fixed form *source* $\rightarrow$ *relation* $\rightarrow$ *target*, without free-form paraphrasing. The resulting triples are supplied alongside the original question, and the model is instructed to rely exclusively on this evidence and return a JSON array containing complete entity names. If the evidence does not support an answer, the expected output is an empty array. Valid predictions are normalized, deduplicated, and evaluated as sets, while failed calls and invalid generations remain in the evaluation as unsuccessful answers.

4 Experimental Setup

We conduct the experiments on the prepared WebQuestionsSP (WebQSP) dataset, which contains Freebase-based questions and their labeled answers (tau Yih et al., 2016). The available data comprise 2,848 training, 250 validation, and 1,639 test questions. Following the recorded evaluation protocol, the validation and test splits are combined into a pool of 1,889 questions. From this pool, 143 questions whose local graphs do not contain any labeled answer are excluded, leaving 1,746 questions on which all configurations are evaluated.

All retrievers are trained for 10 epochs using the Adam optimizer, a learning rate of 10^{-3} , zero weight decay, and batches containing a single graph. Each experimental configuration is evaluated across three independent runs. Architectures that require external embeddings use `text-embedding-3-small`, whereas ReaRev relies on its frozen `all-MiniLM-L6-v2` encoder. A single fixed configuration is used for each architecture, as detailed in Appendix A.

The evidence-construction evaluation compares the union of shortest paths with the two PCST cost formulations for $\lambda \in \{0.01, 0.25, 0.5, 1\}$. The final generation experiment focuses on the endpoint values $\lambda \in \{0.01, 1\}$ alongside the shortest-path baseline, evaluating each evidence configuration with DeepSeek-V4-Flash and GPT-5.6 Luna.

All results are reported as means and sample standard deviations across three independent runs. The metrics are organized to follow how answer information moves through the pipeline. Retrieval evaluation measures both ranking quality and answer coverage using Hits@1, Hits@10, binary-relevance nDCG@10, and macro gold-answer coverage. Evidence-construction metrics then examine how the retrieved candidates are transformed into context by measuring its triple count, candidate retention, and gold-entity coverage. For final generation, question-level Hit and Exact Match capture answer success and completeness, while micro-averaged precision, recall, and F1 assess the quality of the predicted answer sets. Input and output token totals complement these quality measures by quantifying the resource implications of each evidence strategy. Finally, context-utilization outcomes connect the evidence and generation stages by determining whether the model returns all gold answers when they are fully available in the supplied context. Complete definitions of all metrics are provided in Appendix B.

5 Results

5.1 Retrieval architecture comparison

Table 1 reports the results from the six retrievers. NBFNet has the highest mean on every displayed metric: Hits@1 is 0.692 ± 0.015 , Hits@10 is 0.921 ± 0.005 , and nDCG@10 is 0.796 ± 0.007 . Its gold-answer coverage is 0.877 ± 0.005 , and all gold answers are retrieved for 0.827 ± 0.006 of evaluated questions. These jointly indicate strong top-rank placement and coverage of multi-answer questions, with little variation across the three runs.

Table 1: Retrieval results on 1,746 questions, mean $\pm$ sample standard deviation across three runs. RGC is RetrievalGoldCoverage; RFGC is RetrievalFullGoldCoverage. Higher is better for every column. Bold indicates the highest mean, without implying statistical significance.

Architecture	Hits@1	Hits@10	nDCG@10	RGC	RFGC
GraphSAGE	0.046 $\pm$ 0.015	0.252 $\pm$ 0.034	0.087 $\pm$ 0.018	0.187 $\pm$ 0.039	0.129 $\pm$ 0.031
Advance GraphSAGE	0.225 $\pm$ 0.041	0.801 $\pm$ 0.019	0.451 $\pm$ 0.047	0.744 $\pm$ 0.029	0.644 $\pm$ 0.036
R-GCN	0.121 $\pm$ 0.076	0.447 $\pm$ 0.201	0.219 $\pm$ 0.134	0.393 $\pm$ 0.200	0.314 $\pm$ 0.178
HGT	0.221 $\pm$ 0.004	0.671 $\pm$ 0.016	0.383 $\pm$ 0.009	0.613 $\pm$ 0.014	0.518 $\pm$ 0.017
ReaRev	0.588 $\pm$ 0.103	0.848 $\pm$ 0.084	0.701 $\pm$ 0.101	0.771 $\pm$ 0.088	0.706 $\pm$ 0.087
NBFNet	0.692 $\pm$ 0.015	0.921 $\pm$ 0.005	0.796 $\pm$ 0.007	0.877 $\pm$ 0.005	0.827 $\pm$ 0.006

ReaRev is second on the principal ranking measures, with Hits@1 of 0.588 ± 0.103 and nDCG@10 of 0.701 ± 0.101 , but its variability is substantially larger. Its mean Hits@10 of 0.848 exceeds Advance GraphSAGE’s 0.801, while the latter is more consistent on this measure. The result favors NBFNet for the subsequent stages because it combines the strongest observed means with stable retrieval.

Advance GraphSAGE substantially outperforms the GraphSAGE baseline, increasing Hits@10 from 0.252 to 0.801 and gold-answer coverage from 0.187 to 0.744. Taken together, the learned question–relation weights, reverse edges, residual normalization, and question-conditioned classifier therefore provide an effective extension of the baseline. However, because these modifications are evaluated jointly, the observed improvement can be attributed only to the combined design and not to the gate or any other individual component.

Among the explicitly relation-aware architectures, R-GCN is the least stable, achieving a mean Hits@10 of 0.447 with a standard deviation of 0.201. HGT exhibits considerably lower variability, yet still trails Advance GraphSAGE in both Hits@10 and gold-answer coverage. Explicit relation-specific transformations or attention are therefore insufficient, by themselves, to produce stronger retrieval in the evaluated configurations. This comparison must also account for differences in model inputs: unlike the strongest configurations, the implemented R-GCN and HGT variants do not directly condition their predictions on the question. RQ1 consequently concerns the relative performance of these specific implementations under the shared experimental protocol, rather than establishing a general ranking of architecture families independently of their inputs, configurations, and training choices.

5.2 Evidence size and information preservation

The shortest-path union provides the high-coverage reference point for the evidence-construction comparison. Using the same NBFNet predictions, it produces 94.47 ± 22.58 triples and 32.12 ± 6.60 distinct nodes per question while retaining every retrieved candidate under the recorded construction metric. This evidence achieves a ContextGoldCoverage of 0.888 ± 0.006 and full gold-answer coverage for 0.838 ± 0.007 of the questions. Its gold coverage is slightly higher than the retriever’s value of 0.877 because the connecting paths may expose gold entities that were not themselves selected as candidates.

Against this baseline, PCST changes the balance between evidence size and entity coverage substantially. As shown in Figure 2, all evaluated PCST configurations reduce the context to 13.50–14.45 triples, approximately 85–86% fewer than the shortest-path union. Much of this compression occurs without removing candidates. In particular, constant-cost PCST at $\lambda = 0.01$ retains every candidate and achieves a ContextGoldCoverage of 0.881 ± 0.004 , only about 0.7 percentage points below the shortest-path baseline. The reduction therefore comes primarily from selecting fewer connecting facts rather than from discarding retrieved entities. The complete results are reported in Appendix C.

Differences between the cost formulations become clearer at the largest cost scale. With constant costs and $\lambda = 1$, candidate retention falls to 0.949 ± 0.003 , corresponding to a pooled reduction of $5.02 \pm 0.33\%$, while full context coverage decreases to 0.816 ± 0.005 . At the same value of λ , semantic PCST retains 0.991 ± 0.004 of the candidates and reaches full context coverage of 0.830 ± 0.006 with only 13.50 ± 0.24 triples. Semantic

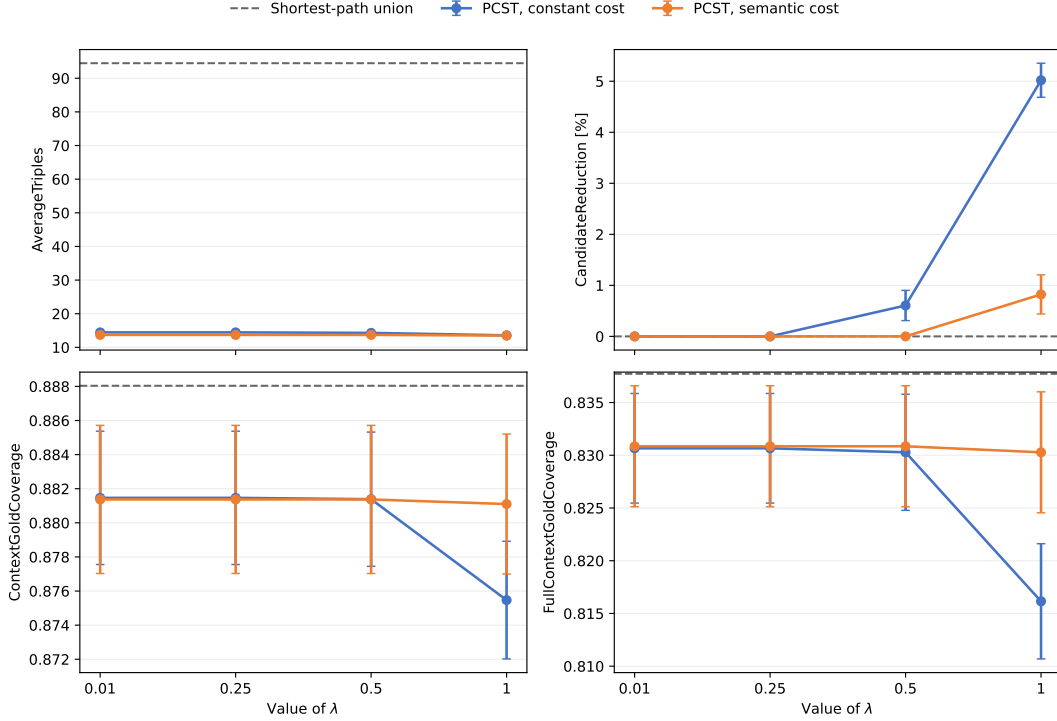


Figure 2: Sensitivity of evidence construction to the PCST cost scale. Constant and semantic costs use the same NBFNet retrievers and four values of λ . Error bars represent standard deviation across three seeds. The accompanying table in Appendix C reports every configuration and metric.

costs therefore preserve more of the retrieved candidate set at the high-cost endpoint without sacrificing the compactness of the resulting evidence.

Across the lower cost scales, however, the aggregate results remain largely stable. Constant costs produce identical reported aggregates at $\lambda = 0.01$ and $\lambda = 0.25$, while the semantic configurations are identical at $\lambda = 0.01$, $\lambda = 0.25$, and $\lambda = 0.5$. This equality does not imply that every individual evidence graph is identical or globally minimal, but it does indicate limited sensitivity of the measured outcomes within this range. Taken together, these results answer RQ2 by showing that PCST can remove most connecting triples while preserving entity-level coverage to a large extent, although stronger constant-cost penalties begin to reduce both candidate retention and full coverage. The next stage examines whether this compact evidence remains equally useful for final-answer generation.

5.3 Final answer quality and token use

Figure 3 presents the answer-quality and token trade-off. Shortest paths produce the strongest final results for both language models. DeepSeek-V4-Flash reaches Hit 0.802 ± 0.005 and F1 0.627 ± 0.008 , while GPT-5.6 Luna reaches Hit 0.804 ± 0.015 and F1 0.613 ± 0.006 . Although their Hit means are close, DeepSeek has higher precision, recall, F1, and Exact Match in this condition. A question-level hit therefore hides differences in how completely and selectively the models return multi-answer sets.

Constant-cost PCST causes the largest quality reduction. At $\lambda = 1$, Hit falls to 0.594 ± 0.014 for DeepSeek and 0.645 ± 0.015 for GPT, which is a decrease of approximately 20.8 and 15.9 percentage points from shortest paths. Both models reach F1 of approximately 0.398. Lowering λ to 0.01 offers only small changes, although the low-cost construction retains every candidate. Candidate omission alone therefore cannot explain the observed quality gap.

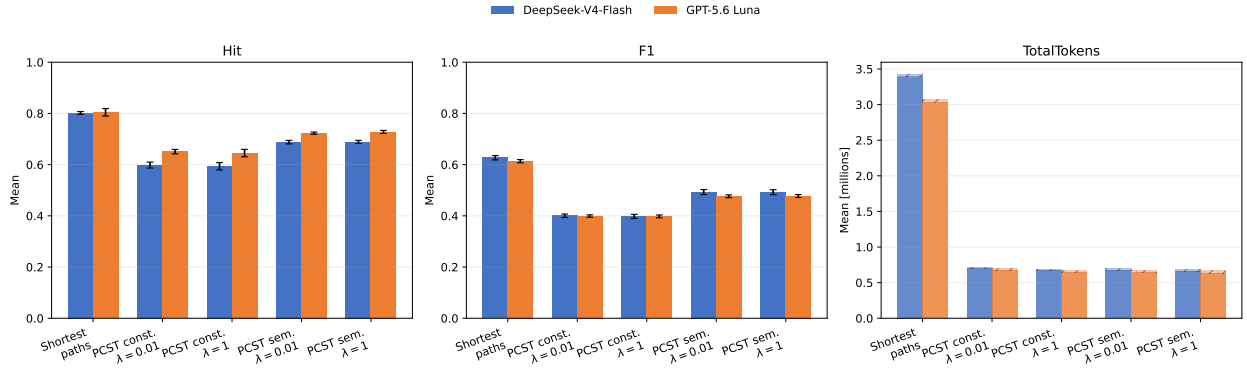


Figure 3: Final Hit, micro F1, and total tokens for the two language models. Within each panel, configurations run from shortest paths to constant PCST ($\lambda = 0.01, 1$) and semantic PCST ($\lambda = 0.01, 1$). In the token panel, the main bar segment represents input tokens and the hatched segment output tokens. Values are means with standard deviations across three runs; tokens are in millions.

Among the compact evidence strategies, semantic PCST provides the stronger alternative. At $\lambda = 1$, it achieves Hit scores of 0.689 ± 0.006 for DeepSeek and 0.728 ± 0.006 for GPT, with corresponding F1 scores of 0.492 ± 0.010 and 0.477 ± 0.006 . These results are similar to those obtained with semantic PCST at $\lambda = 0.01$, while both semantic configurations perform substantially better than their constant-cost counterparts. Within the tested range, final-answer quality therefore appears to depend more on the definition of the edge costs than on the choice between the two endpoint values of λ .

The shorter evidence contexts also lead to substantial reductions in token usage. Shortest-path runs consume 3.42 ± 0.77 million tokens with DeepSeek and 3.07 ± 0.66 million with GPT, whereas the PCST configurations require approximately 0.66–0.72 million tokens, corresponding to relative reductions of about 78–80% within each model. For example, GPT with semantic PCST at $\lambda = 1$ uses approximately 4.6 times fewer tokens than its shortest-path counterpart, but its Hit decreases by about 7.6 percentage points and its F1 falls from 0.613 to 0.477. Compact evidence therefore offers a substantial efficiency gain, but not equivalent answer quality: the reduction in resource usage is accompanied by a measurable loss in performance. Appendix D reports the complete quality and token results, while the following analysis examines where this performance loss arises.

5.4 Context utilization and answer omissions

To separate answer availability in the evidence from the language model’s ability to use that evidence, we consider three complementary measures. FullContextCompleteAnswer captures the fraction of all evaluated questions for which the evidence contains every gold answer and the model successfully returns all of them. Using the same overall denominator, FullContextLLMOmission instead captures cases in which the evidence is complete but the model omits at least one gold answer. Finally, LLMOMissionGivenFullContext isolates the latter behavior by restricting the denominator to questions whose evidence contains every gold answer. An omission is defined only by the absence of a gold answer, therefor returning additional incorrect answers does not count as an omission, although it may prevent Exact Match. Complete definitions are provided in Appendix B.

Figure 4 shows that answer omissions persist even with high context coverage. For shortest paths, FullContextCompleteAnswer is 0.661 ± 0.004 with DeepSeek and 0.639 ± 0.003 with GPT, whereas their conditional omission rates are 0.211 ± 0.002 and 0.237 ± 0.010 . Thus, even when every gold entity occurs in the evidence, at least one is missing from the generated answer in approximately one fifth to one quarter of questions.

The discrepancy becomes larger after compression. Constant PCST at $\lambda = 1$ has conditional omission rates of 0.587 ± 0.007 and 0.573 ± 0.014 for DeepSeek and GPT. Semantic PCST improves these to 0.477 ± 0.003 and 0.474 ± 0.005 , respectively, but remains well above shortest paths. Meanwhile, full context coverage for

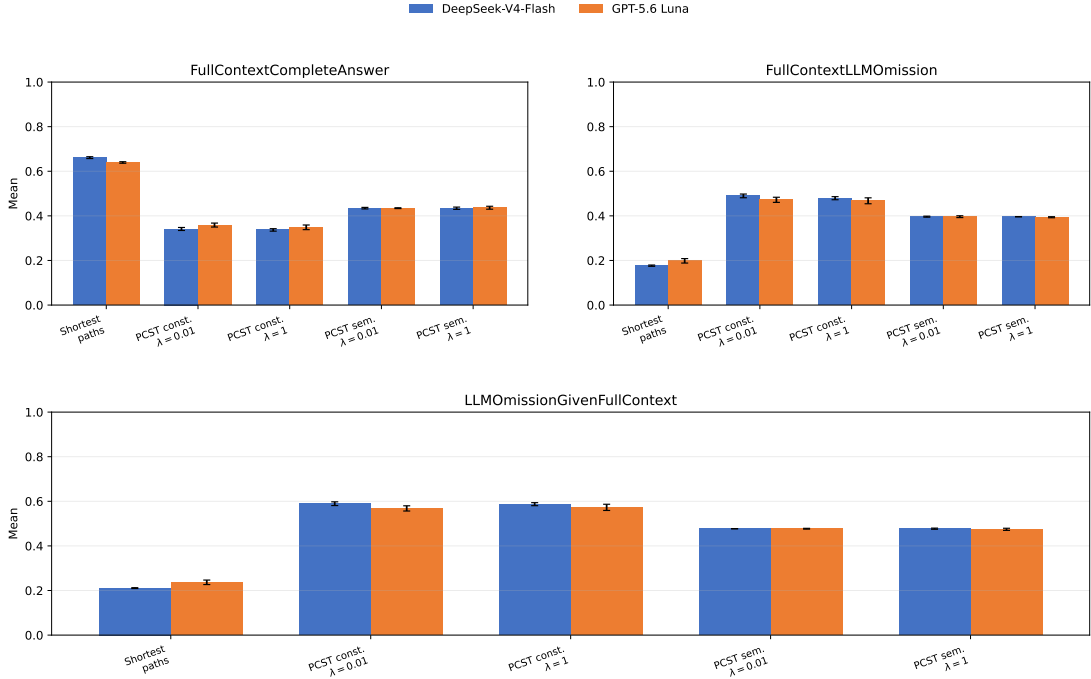


Figure 4: Use of gold answers present in the evidence. The top panels show complete answers and omissions as fractions of all eligible questions; the bottom panel shows omissions conditional on full context coverage. Configuration order is the same as Figure 3. Error bars indicate standard deviation across three runs. These measures test whether gold entities are returned, not whether the full set of predicted entities is exact.

semantic PCST is still 0.830 ± 0.006 . The principal change is therefore not a large disappearance of answer entities from the context, but a reduced ability of the evidence-to-generation stage to return them all.

With respect to RQ3, these results show that answer omissions cannot be attributed solely to the language model. A gold entity may remain visible in the context while the surrounding evidence fails to establish why it answers the question. Compression may remove alternative paths or relational qualifications that help the model interpret the entity’s role, although the aggregate measurements cannot identify which missing facts are responsible for an individual failure. The supported conclusion is therefore narrower but important: neither graph compactness nor answer-entity presence alone is a sufficient indicator of downstream evidence usefulness.

6 Discussion

6.1 What the architecture comparison establishes

The observed ranking is consistent with the value of question-conditioned reasoning over paths. NBFNet and ReaRev achieve the strongest retrieval results, while a learned question-aware extension substantially improves the aggregation baseline. However, the experiment does not isolate question conditioning from model capacity, loss design, or representation choice. The evaluated configurations differ in hidden width, encoding, and normalization as well as in message passing. Their results are best interpreted as an assessment of practical implementations within a shared task and budget. This distinction matters when relating the study to earlier methods. The results support the usefulness of an independently implemented GNN-guided workflow and expose substantial architectural sensitivity.

6.2 Compact graphs are not necessarily sufficient evidence

PCST and the shortest-path union preserve different aspects of the graph. Whereas the shortest-path union retains every minimal route connecting the candidates to the seed set, PCST favors a low-cost shared structure and may remove alternative relational routes even when their endpoint candidates remain selected. The low-cost constant PCST result illustrates the importance of this distinction: although every candidate is retained, final-answer quality still declines substantially. Candidate retention alone therefore cannot explain the advantage of the larger shortest-path context.

Semantic costs recover part of the quality gap while preserving a similar triple budget. This is compatible with question–relation similarity selecting more useful connections, although it is not a direct semantic correctness test of individual edges. In particular, similarity does not guarantee that a set of triples jointly entails an answer. A relation that appears only weakly related to the question in isolation may still be essential to an intermediate step. The results motivate treating the sufficiency of relational evidence as a separate objective from node coverage and graph size.

6.3 Why intermediate evaluation matters

Information does not pass through the workflow as a strictly decreasing sequence of answer sets. During evidence construction, paths connecting seeds and candidates may introduce gold-answer entities that were not present in the retrieved candidate list, allowing context gold coverage to exceed retrieval coverage slightly. The opposite effect can occur during generation: a gold entity may be present in the evidence but absent from the final answer. Evaluating retrieval or final accuracy in isolation would conceal these distinct transitions and make it difficult to determine where answer information is gained or lost.

The conditional omission metric provides a more appropriate view of the downstream transition. Retrieval and context coverage are macro-averaged across questions, whereas final recall is computed by pooling answer counts. Subtracting these quantities would therefore not yield a valid stage-specific loss estimate. Instead, `LLMOmissionGivenFullContext` considers only questions for which every gold answer is available in the evidence. In these cases, missing answer entities cannot by themselves explain an incomplete prediction. The metric does not identify the precise cause of the omission, which may involve insufficient relational support, misinterpretation by the generator, normalization failures for equivalent names, or unsuccessful generation, but it isolates the stage at which further analysis is needed.

Under this evaluation, therefore, the transition from evidence to final answer emerges as a substantial bottleneck. After evidence compression, both language models frequently omit gold answers even when those answers remain available in the context, showing that entity coverage does not reliably translate into answer completeness. Their behavior also differs across the evaluated strategies: GPT attains higher Hit on compact contexts, while DeepSeek preserves higher precision and F1 under the semantic-cost configurations. These differences cannot be represented adequately by a single score. Assessing answer completeness, selectivity, and resource usage together therefore provides a more informative basis for comparing and selecting system configurations.

7 Limitations

The study uses one dataset and supplied local graphs with given seed entities. Excluding graphs without all labeled answers conditions the findings on upstream graph completeness and removes entity-linking failures from consideration. The combined validation/test evaluation and downstream selection of NBFNet on that evaluation pool means that these numbers are comparative results under this protocol, not an independently held-out estimate for a selected system or a directly comparable WebQSP leaderboard result.

Additionally, only one configuration per architecture and three random runs are evaluated. Architectures differ in question access, embeddings, capacity, and objective, so the comparison does not attribute improvements to individual components or establish statistical significance. Some methods are adapted from tasks other than KGQA and downstream experiments use NBFNet only, leaving interactions between other retrievers and evidence builders unmeasured.

The downstream generation evaluation introduces a separate set of scope constraints. It compares two model configurations using the same answer-only prompt, while the three repetitions inherit different retriever runs and their corresponding evidence contexts. Consequently, these repetitions capture end-to-end variation rather than generator randomness under identical inputs. Token totals remain useful for comparing evidence strategies within each model, but they do not support direct comparisons of financial cost or latency across providers. Similarly, context-coverage metrics establish whether gold entities occur in the evidence, not whether the selected relations provide logically sufficient support for deriving the answer. The analysis can therefore identify the evidence-to-answer transition as a key source of performance loss, but it cannot determine the precise cause of each omission or isolate the contribution of the generator from that of the supplied evidence.

8 Conclusion

This study reassesses GNN-guided knowledge graph question answering through its retrieval, evidence-construction, and generation stages. Among the six evaluated retrievers, NBFNet achieves the strongest mean ranking and coverage results with low variation across runs. The comparison also shows that an extended question-aware GraphSAGE implementation can improve substantially over the weighted aggregation baseline, while relation-specific modeling alone is insufficient to ensure stronger retrieval in the tested configurations.

Rooted PCST compresses evidence by approximately 85% and reduces token use by approximately 78–80%, largely preserving answer-entity coverage. This compression nevertheless lowers final answer quality. Semantic costs provide the stronger compact alternative, but shortest-path union remains the best-performing evidence strategy for both language models. Conditional omissions demonstrate that many correct entities present in the context are still not returned, especially after compression.

The resulting design lesson is to assess evidence through what it enables the generator to answer, as well as through what it contains. Future work can investigate hybrid evidence selection that preserves informative paths, objectives connected to answer completeness, and joint optimization of retrieval and evidence construction. Such extensions should be evaluated across additional datasets and configurations. Finally, effective GNN-guided answering requires candidate relevance, sufficient relational evidence, and successful context use to be considered together.

References

- Gabriele Corso, Luca Cavalleri, Dominique Beaini, Pietro Liò, and Petar Veličković. Principal neighbourhood aggregation for graph nets. In *Advances in Neural Information Processing Systems*, volume 33, pp. 13260–13271, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/99cad265a1768cc2dd013f0e740300ae-Abstract.html>.
- Michel X. Goemans and David P. Williamson. A general approximation technique for constrained forest problems. *SIAM Journal on Computing*, 24(2):296–317, 1995. doi: 10.1137/S0097539793242618. URL <https://doi.org/10.1137/S0097539793242618>.
- William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, volume 30, pp. 1024–1034, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/5dd9db5e033da9c6fb5ba83c7a7e9bea9-Abstract.html>.
- Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V. Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/efaf1c9726648c8ba363a5c927440529-Abstract.html.
- Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutiérrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine

- Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4):71:1–71:37, 2021. doi: 10.1145/3447772. URL <https://doi.org/10.1145/3447772>.
- Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. Heterogeneous graph transformer. In *Proceedings of The Web Conference 2020*, pp. 2704–2710. ACM, 2020. doi: 10.1145/3366423.3380027. URL <https://doi.org/10.1145/3366423.3380027>.
- Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002. doi: 10.1145/582415.582418. URL <https://doi.org/10.1145/582415.582418>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- Costas Mavromatis and George Karypis. ReaRev: Adaptive reasoning for question answering over knowledge graphs. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pp. 2447–2458, Abu Dhabi, United Arab Emirates, 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.findings-emnlp.181. URL <https://aclanthology.org/2022.findings-emnlp.181/>.
- Costas Mavromatis and George Karypis. GNN-RAG: Graph neural retrieval for efficient large language model reasoning on knowledge graphs. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 16682–16699, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-acl.856. URL <https://aclanthology.org/2025.findings-acl.856/>.
- Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *The Semantic Web*, volume 10843 of *Lecture Notes in Computer Science*, pp. 593–607. Springer, 2018. doi: 10.1007/978-3-319-93417-4_38. URL https://doi.org/10.1007/978-3-319-93417-4_38.
- Wen tau Yih, Matthew Richardson, Christopher Meek, Ming-Wei Chang, and Jina Suh. The value of semantic parse labeling for knowledge base question answering. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 201–206, Berlin, Germany, 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-2033. URL <https://aclanthology.org/P16-2033/>.
- Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems*, volume 33, pp. 5776–5788, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
- Bishan Yang, Wen tau Yih, Xiaodong He, Jianfeng Gao, and Li Deng. Embedding entities and relations for learning and inference in knowledge bases. In *International Conference on Learning Representations*, 2015. URL <https://arxiv.org/abs/1412.6575>.
- Zhaocheng Zhu, Zuobai Zhang, Louis-Pascal A. C. Xhonneux, and Jian Tang. Neural Bellman-Ford networks: A general graph neural network framework for link prediction. In *Advances in Neural Information Processing Systems*, volume 34, pp. 29476–29490, 2021. URL <https://proceedings.neurips.cc/paper/2021/hash/f6a673f09493afcd8b129a0bcf1cd5bc-Abstract.html>.

A Architecture and training details

Table 2 summarizes the architecture configurations specified in the current experiment manifest. Because the models differ in width and internal computation, a common training duration does not imply equal parameter counts or computational requirements. Throughout the comparison, GraphSAGE refers to the weighted baseline described in Section 3.2, while Advance GraphSAGE denotes the combined extension evaluated in this study.

Table 2: Fixed retrieval configurations. “3/3/3” denotes ReaRev’s instructions, reasoning steps, and adaptive iterations, respectively. Dropout 0.1 is explicitly configured for the first five models; NBFNet uses its fixed path-propagation implementation.

Architecture	Width	Depth	Main configured characteristics
GraphSAGE	256	3	Fixed cosine edge weights; MLP node classifier.
Advance GraphSAGE	256	3	Learned edge MLP (width 256), reverse edges, residual normalization, question-aware classifier.
R-GCN	256	3	30 relation bases; categorical forward/reverse relations.
HGT	128	3	8 attention heads; one entity node type; relation-dependent attention.
ReaRev	50	3/3/3	Frozen MiniLM; token instructions; adaptive revision.
NBFNet	32	3	Question-conditioned boundary states; DistMult messages; PNA aggregation.

Aggregation variants. The aggregation variants differ primarily in how they incorporate relation and question information into message passing. For each incoming triple (u, r, v) , the GraphSAGE baseline transforms the source state, weights it by the cosine similarity between the question and relation embeddings, and sums the resulting messages. This aggregate is concatenated with the current target state before the learned node update. Advance GraphSAGE extends this mechanism with the gate from Equation 2 and the residual-normalized update

$$\mathbf{h}_v^{(\ell+1)} = \text{ReLU}\left(\text{LayerNorm}\left(\mathbf{h}_v^{(\ell)} + \text{Dropout}\left(\mathbf{u}_v^{(\ell)}\right)\right)\right),$$

where $\mathbf{u}_v^{(\ell)}$ denotes the message-passing update.

R-GCN and HGT instead encode relation types directly within their aggregation operators. R-GCN parameterizes each relation transformation through the 30-basis decomposition

$$W_r^{(\ell)} = \sum_{b=1}^{30} a_{rb}^{(\ell)} V_b^{(\ell)},$$

applies relation-wise neighbor normalization, and assigns distinct identifiers to forward and reverse relations. HGT uses relation-specific attention and message transformations across eight attention heads, together with shared node-type projections, a learned residual gate, and normalization.

Task-oriented models. ReaRev uses the frozen encoder `sentence-transformers/all-MiniLM-L6-v2`, with maximum question and relation lengths of 64 and 16 tokens. Token-level attention produces three instructions. Initial entity states derive from incident relation information, and reasoning begins from a seed distribution. Instructions are revised after each reasoning iteration. NBFNet, on the other hand, supplies the full projected question vector to each seed, uses separate original and reverse relation operators, and reinjects the initial boundary condition at each layer. PNA combines mean, maximum, minimum, and standard deviation statistics with degree scaling (Corso et al., 2020). Final node states are concatenated with the question representation for scoring.

Training objective. All models except ReaRev use independent binary answer labels and class-balanced binary cross-entropy. For a graph with n_+ positive and n_- negative nodes, the positive term receives weight

n_-/n_+ when $n_+ > 0$. ReaRev instead minimizes KL divergence to a distribution uniform over the gold answer nodes. Independent outputs are passed through sigmoid, whereas ReaRev yields a softmax distribution. The common candidate-selection rule is applied after these normalizations. The model comparison therefore includes differences in objective and calibration, as well as architecture.

B Metric definitions and evaluation protocol

Let Q be the set of evaluated questions with nonempty normalized gold sets and $N = |Q| = 1746$. Let $C_q^{(k)}$ be the top k retrieved candidates, C_q the final candidate set, U_q the source and target entities actually appearing in emitted evidence triples, and P_q the normalized prediction set. The indicator function $\mathbb{I}[\cdot]$ is one when its argument holds. Reported standard deviations are computed over the three run-level values, using denominator $3 - 1$.

Retrieval. Retrieval is evaluated from two complementary perspectives. Hits@ k measures whether at least one gold answer appears among the first k candidates, while nDCG@ k additionally rewards gold answers that occur closer to the top of the ranking and normalizes the score against an ideal ordering (Järvelin & Kekäläinen, 2002):

$$\text{Hits@}k = \frac{1}{N} \sum_{q \in Q} \mathbb{I}[A_q \cap C_q^{(k)} \neq \emptyset], \quad \text{nDCG@}k = \frac{1}{N} \sum_{q \in Q} \frac{\sum_{i=1}^k \text{rel}_{qi} / \log_2(i+1)}{\sum_{i=1}^{\min(|A_q|, k)} 1 / \log_2(i+1)}. \quad (5)$$

Here, relevance is binary: $\text{rel}_{qi} = 1$ when the candidate at position i belongs to A_q , and 0 otherwise. Candidate positions that do not exist also contribute zero.

The coverage metrics reported in Table 1 capture how much of the complete gold-answer set is retrieved. RetrievalGoldCoverage (RGC) measures the average fraction of gold answers included in the candidate list, whereas RetrievalFullGoldCoverage (RFGC) measures the fraction of questions for which every gold answer is retrieved:

$$\text{RGC} = \frac{1}{N} \sum_{q \in Q} \frac{|A_q \cap C_q|}{|A_q|}, \quad \text{RFGC} = \frac{1}{N} \sum_{q \in Q} \mathbb{I}[A_q \subseteq C_q]. \quad (6)$$

Both are macro-averaged across questions, so each question contributes equally regardless of the size of its gold-answer set.

Evidence. Evidence construction is evaluated in terms of context size, candidate retention, and gold-answer visibility. Let U_q denote the distinct nodes appearing in the selected triples H_q . AverageTriples and AverageDistinctNodes measure the resulting context size:

$$\text{AverageTriples} = \frac{1}{N} \sum_{q \in Q} |H_q|, \quad \text{AverageDistinctNodes} = \frac{1}{N} \sum_{q \in Q} |U_q|.$$

To evaluate candidate retention, let $D_q \subseteq C_q$ be the retrieved candidates retained by the construction procedure. ContextCandidateCoverage macro-averages the retained fraction $|D_q|/|C_q|$, assigning zero when C_q is empty. CandidateReduction instead pools candidate counts across questions:

$$\text{CandidateReduction} = 100 \frac{\sum_{q \in Q} (|C_q| - |D_q|)}{\sum_{q \in Q} |C_q|}.$$

ContextGoldCoverage (CGC) and ContextFullGoldCoverage (CFG) measure gold-answer visibility by replacing C_q with U_q in the RGC and RFGC definitions. This distinction is important because retaining a candidate does not necessarily make it visible to the language model: a seed-only structure with no selected edge produces no serialized triple containing that entity.

Final-answer quality. Final-answer quality is evaluated at both the question and individual-answer levels. At the question level, Hit measures whether the prediction contains at least one gold answer, whereas Exact Match (EM) requires the predicted and gold-answer sets to be identical:

$$\text{Hit} = \frac{1}{N} \sum_{q \in Q} \mathbb{I}[A_q \cap P_q \neq \emptyset], \quad \text{EM} = \frac{1}{N} \sum_{q \in Q} \mathbb{I}[A_q = P_q]. \quad (7)$$

Precision, recall, and F1 are instead computed from answer counts pooled across all questions. The aggregate numbers of true positives, false positives, and false negatives are

$$\text{TP} = \sum_{q \in Q} |A_q \cap P_q|, \quad \text{FP} = \sum_{q \in Q} |P_q \setminus A_q|, \quad \text{FN} = \sum_{q \in Q} |A_q \setminus P_q|. \quad (8)$$

The corresponding micro-averaged metrics are

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{F1} = \frac{2 \text{ Precision Recall}}{\text{Precision} + \text{Recall}}. \quad (9)$$

A metric is assigned zero whenever its denominator is zero. Failed generations are represented by an empty prediction set and are explicitly treated as Exact Match failures. Unlike Hit and Exact Match, which weight each question equally, the pooled metrics weight individual answer instances.

Context utilization. To distinguish missing evidence from failures to use available evidence, we define two question-level indicators. Let

$$F_q = \mathbb{I}[A_q \subseteq U_q]$$

indicate that every gold answer is present in the evidence context, and let

$$O_q = \mathbb{I}[A_q \not\subseteq P_q]$$

indicate that the final prediction omits at least one gold answer. Their combination defines two outcomes for fully covered questions, together with a conditional omission rate:

$$\text{FullContextCompleteAnswer} = \frac{1}{N} \sum_{q \in Q} F_q(1 - O_q), \quad (10)$$

$$\text{FullContextLLMOmission} = \frac{1}{N} \sum_{q \in Q} F_q O_q, \quad (11)$$

$$\text{LLMOmissionGivenFullContext} = \frac{\sum_{q \in Q} F_q O_q}{\sum_{q \in Q} F_q}. \quad (12)$$

The first two metrics use all evaluated questions as their denominator and partition the questions with full context coverage. Consequently, their sum equals CFGC for each run rather than one. LLMOmissionGivenFullContext instead restricts the denominator to fully covered questions, directly measuring how often the model omits a gold answer despite having all gold entities available in the context. The metric is undefined when no question has full context coverage, although this case does not occur in the reported results. Answer completeness requires every gold answer to be returned but permits additional predictions, unlike Exact Match.

C Complete evidence-construction results

Table 3 summarizes all nine evidence-construction settings, each of which reuses the same three NBFNet runs. PCST is applied to a multigraph containing the original facts, without adding synthetic inverse facts to the resulting evidence. When a question has multiple seeds, every seed receives an auxiliary prize exceeding the combined candidate prizes and structural edge costs, and is linked to a temporary root through a zero-cost edge. Removing this root after optimization may produce several seed-anchored branches. These auxiliary enforcement prizes are excluded from both the stated candidate-prize definition and the reported collected-prize values.

Table 3: Evidence results, mean $\pm$ standard deviation across three NBFNet runs. CCC: macro candidate coverage; Red.: pooled candidate reduction in percent; CGC: context gold coverage; CFGC: full context gold coverage. Constant and semantic rows differ only in their edge-cost definition and scale.

Strategy	λ	Triples	Nodes	CCC	Red. (%)	CGC	CFGC
Shortest paths	–	94.47 $\pm$ 22.58	32.12 $\pm$ 6.60	1.000 $\pm$ 0.000	0.00 $\pm$ 0.00	0.888 $\pm$ 0.006	0.838 $\pm$ 0.007
PCST constant	0.01	14.45 $\pm$ 0.26	15.46 $\pm$ 0.26	1.000 $\pm$ 0.000	0.00 $\pm$ 0.00	0.881 $\pm$ 0.004	0.831 $\pm$ 0.005
	0.25	14.45 $\pm$ 0.26	15.46 $\pm$ 0.26	1.000 $\pm$ 0.000	0.00 $\pm$ 0.00	0.881 $\pm$ 0.004	0.831 $\pm$ 0.005
	0.5	14.30 $\pm$ 0.25	15.31 $\pm$ 0.25	0.994 $\pm$ 0.003	0.61 $\pm$ 0.30	0.881 $\pm$ 0.004	0.830 $\pm$ 0.006
	1	13.57 $\pm$ 0.26	14.58 $\pm$ 0.26	0.949 $\pm$ 0.003	5.02 $\pm$ 0.33	0.875 $\pm$ 0.003	0.816 $\pm$ 0.005
PCST semantic	0.01	13.70 $\pm$ 0.28	14.71 $\pm$ 0.28	1.000 $\pm$ 0.000	0.00 $\pm$ 0.00	0.881 $\pm$ 0.004	0.831 $\pm$ 0.006
	0.25	13.70 $\pm$ 0.28	14.71 $\pm$ 0.28	1.000 $\pm$ 0.000	0.00 $\pm$ 0.00	0.881 $\pm$ 0.004	0.831 $\pm$ 0.006
	0.5	13.70 $\pm$ 0.28	14.71 $\pm$ 0.28	1.000 $\pm$ 0.000	0.00 $\pm$ 0.00	0.881 $\pm$ 0.004	0.831 $\pm$ 0.006
	1	13.50 $\pm$ 0.24	14.51 $\pm$ 0.24	0.991 $\pm$ 0.004	0.82 $\pm$ 0.38	0.881 $\pm$ 0.004	0.830 $\pm$ 0.006

D Complete final results and reproducibility details

Table 4 summarizes all ten model–evidence combinations, covering final-answer quality, token usage, and the conditional omission rates discussed in Section 5.4. Each row aggregates the same three NBFNet retriever runs.

Table 4: Complete final-answer and context-utilization results. P and R denote micro-averaged precision and recall, EM denotes Exact Match, and Omission denotes LLMOmissionGivenFullContext; lower omission is better. Tokens are the total input and output tokens in millions per run. SP denotes the shortest-path union, C constant-cost PCST, and S semantic-cost PCST. All entries are mean $\pm$ standard deviation across three runs.

Evidence	Hit	P	R	F1	EM	Omission	Tokens (M)
DeepSeek-V4-Flash							
SP	0.802 $\pm$ 0.005	0.860 $\pm$ 0.009	0.493 $\pm$ 0.007	0.627 $\pm$ 0.008	0.580 $\pm$ 0.003	0.211 $\pm$ 0.002	3.42 $\pm$ 0.77
C, $\lambda = 0.01$	0.598 $\pm$ 0.012	0.881 $\pm$ 0.006	0.259 $\pm$ 0.005	0.400 $\pm$ 0.007	0.322 $\pm$ 0.002	0.589 $\pm$ 0.008	0.72 $\pm$ 0.01
C, $\lambda = 1$	0.594 $\pm$ 0.014	0.884 $\pm$ 0.007	0.257 $\pm$ 0.006	0.398 $\pm$ 0.008	0.320 $\pm$ 0.002	0.587 $\pm$ 0.007	0.69 $\pm$ 0.01
S, $\lambda = 0.01$	0.688 $\pm$ 0.007	0.885 $\pm$ 0.008	0.342 $\pm$ 0.008	0.493 $\pm$ 0.010	0.410 $\pm$ 0.003	0.477 $\pm$ 0.001	0.69 $\pm$ 0.01
S, $\lambda = 1$	0.689 $\pm$ 0.006	0.886 $\pm$ 0.008	0.341 $\pm$ 0.008	0.492 $\pm$ 0.010	0.411 $\pm$ 0.004	0.477 $\pm$ 0.003	0.69 $\pm$ 0.01
GPT-5.6 Luna							
SP	0.804 $\pm$ 0.015	0.846 $\pm$ 0.007	0.481 $\pm$ 0.006	0.613 $\pm$ 0.006	0.571 $\pm$ 0.006	0.237 $\pm$ 0.010	3.07 $\pm$ 0.66
C, $\lambda = 0.01$	0.651 $\pm$ 0.009	0.847 $\pm$ 0.001	0.261 $\pm$ 0.004	0.399 $\pm$ 0.005	0.341 $\pm$ 0.006	0.568 $\pm$ 0.012	0.70 $\pm$ 0.01
C, $\lambda = 1$	0.645 $\pm$ 0.015	0.849 $\pm$ 0.006	0.260 $\pm$ 0.004	0.398 $\pm$ 0.006	0.331 $\pm$ 0.009	0.573 $\pm$ 0.014	0.67 $\pm$ 0.01
S, $\lambda = 0.01$	0.723 $\pm$ 0.004	0.850 $\pm$ 0.008	0.331 $\pm$ 0.004	0.476 $\pm$ 0.005	0.414 $\pm$ 0.003	0.477 $\pm$ 0.002	0.67 $\pm$ 0.01
S, $\lambda = 1$	0.728 $\pm$ 0.006	0.854 $\pm$ 0.007	0.331 $\pm$ 0.006	0.477 $\pm$ 0.006	0.414 $\pm$ 0.005	0.474 $\pm$ 0.005	0.66 $\pm$ 0.01